

Republic of Algeria — Democratic and Popular
Ministry of Higher Education and Scientific Research
University of Science and Technology — Oran (USTO-MB)
Faculty of Computer Science
Department of Computer Science

Design and Implementation of an AI-Driven Orchestration and Decision System for Automated Penetration Testing

Master Thesis — PFE (Projet de Fin d'Études)

Specialty: Computer Science — Cybersecurity and AI Systems

Presented by:

Rodwan Gadouri

Supervised by:

Dr. Allama Oussama

Academic Year: 2025 – 2026

Abstract

Abstract

Penetration testing is a foundational practice in proactive cybersecurity, yet its operational form remains predominantly manual, expensive, and impossible to scale. Automated tools based on signature matching and fixed execution pipelines have proven categorically insufficient against the combinatorial complexity of modern web application attack surfaces.

This thesis presents the design and implementation of PHANTOM, an AI-driven orchestration and decision system for autonomous penetration testing. The system is grounded in the *Reason-Act* (ReAct) paradigm and introduces four original contributions: (1) a *Hypothesis Ledger* implementing structured external memory for belief propagation and payload deduplication across compression cycles; (2) a *Correlation Engine* using Bayesian priors to identify and rank multi-vulnerability chain opportunities; (3) an *Attack Graph* module employing betweenness centrality for critical node identification; and (4) a *hierarchical multi-agent architecture* enabling concurrent, task-partitioned exploration. These contributions are integrated within a seven-layer security model that governs scope enforcement, tool-level access control, cost management, and HMAC-signed tamper-evident audit trails.

The thesis positions PHANTOM as a formally-grounded architecture that bridges the gap between LLM generative reasoning and production-grade penetration testing discipline. By addressing the fundamental divide between stateless agentic execution and the contextual semantic reasoning required to uncover novel, non-templated vulnerabilities, this research provides a robust framework for sequential decision-making in autonomous offensive security.

Keywords: Autonomous Penetration Testing, AI Agents, Large Language Models, Hypothesis-Driven Reasoning, Vulnerability Chaining, Cybersecurity Automation.

Résumé

Résumé

Les tests de pénétration constituent une pratique fondamentale en cybersécurité proactive, mais leur exécution reste largement manuelle, coûteuse et impossible à mettre à l'échelle. Les outils automatisés basés sur la correspondance de signatures et les pipelines d'exécution fixes se sont révélés catégoriquement insuffisants face à la complexité combinatoire des surfaces d'attaque modernes.

Cette thèse présente la conception et l'implémentation de PHANTOM, un système d'orchestration et de décision piloté par l'IA pour les tests de pénétration autonomes. Le système repose sur le paradigme *Reason-Act* (ReAct) et introduit quatre contributions originales : (1) un *Hypothesis Ledger* implémentant une mémoire externe structurée pour la propagation des croyances et la déduplication des charges utiles ; (2) un *Correlation Engine* utilisant des a priori bayésiens pour classer les opportunités de chaînes de vulnérabilités ; (3) un module *Attack Graph* exploitant la centralité d'intermédiarité pour identifier les nœuds critiques ; et (4) une *architecture multi-agents hiérarchique* permettant une exploration concurrente et partitionnée. Ces contributions sont intégrées au sein d'un modèle de sécurité en sept couches qui régit le périmètre, le contrôle d'accès aux outils, la gestion des coûts et les pistes d'audit infalsifiables.

La thèse positionne PHANTOM comme une architecture formellement fondée qui comble le fossé entre le raisonnement génératif des LLM et la rigueur d'un test de pénétration. En abordant la fracture entre l'exécution agentique sans état et le raisonnement sémantique requis pour découvrir de nouvelles vulnérabilités non basées sur des modèles, cette recherche fournit un cadre robuste pour la prise de décision en sécurité offensive autonome.

Mots-clés : Tests d'intrusion autonomes, Agents IA, Grands modèles de langage, Raisonnement par hypothèses, Enchaînement de vulnérabilités, Cybersécurité automatisée.

Contents

Abstract	i
Résumé	ii
Contents	iii
List of Tables	vii
List of Tables	vii
List of Figures	viii
List of Figures	viii
List of Acronyms	ix
1 Introduction	1
1.1 Context and Motivation	1
1.2 Problem Statement	1
1.3 Research Gap	2
1.4 Research Hypotheses	3
1.5 Objectives	3
1.6 Contributions	3
1.7 Thesis Organization	4
2 Background and Foundations	5
2.1 Penetration Testing: Concepts and Methodology	5
2.1.1 Definition and Scope	5
2.1.2 Web Application Attack Taxonomy	6

2.1.3	Vulnerability Chaining	6
2.2	Automated Vulnerability Detection	7
2.2.1	Signature-Based Scanning	7
2.2.2	Static Analysis Approaches	7
2.3	AI and Large Language Models in Cybersecurity	8
2.3.1	Large Language Models: Capabilities and Limitations	8
2.3.2	The ReAct Paradigm	8
2.3.3	Multi-Agent Systems	9
2.4	Memory Systems in AI Agents	9
2.4.1	Conversation History Limitations	9
2.4.2	External Memory and the Hypothesis Framework	9
2.5	Cost Management in LLM Systems	10
3	State of the Art	11
3.1	Overview and Classification	11
3.2	Traditional Penetration Testing Automation	11
3.2.1	Nuclei: Template-Based Vulnerability Detection	11
3.2.2	OWASP ZAP: Active Heuristic Scanning	12
3.2.3	SQLMap: Domain-Specific Exploitation	12
3.3	AI-Based Penetration Testing: Emerging Approaches	12
3.3.1	PentestGPT	12
3.3.2	HackingBuddyGPT	13
3.3.3	AutoAttacker	13
3.3.4	LLM Exploitation Capability: Critical Evidence from Liu et al.	13
3.4	Memory and State Management in Agentic Systems	14
3.5	Synthesis: Research Gap Analysis	14
4	Critical Analysis and Research Positioning	16
4.1	The Statelessness Problem	16
4.1.0.1	Payload Redundancy	16

4.1.0.2	Finding Loss Under Compression	16
4.1.0.3	Cross-Agent Ignorance	16
4.2	Absence of Formal Hypothesis Management	17
4.3	Decision-Making Under Uncertainty: Theoretical Framing	17
4.4	The CVE Template Gap as a Structural Limitation	18
4.5	Positioning the Proposed Framework	18
5	Proposed Conceptual Framework	19
5.1	Design Principles	19
5.2	System Architecture	19
5.3	Contribution 1: The Hypothesis Ledger (Gaps G4, G5)	20
5.4	Contribution 2: The Correlation Engine (Gap G3)	22
5.5	Contribution 3: The Attack Graph Module (Gap G3)	22
5.6	Contribution 4: Hierarchical Multi-Agent Architecture	23
5.7	The Seven-Layer Security Model	23
5.8	Memory Compression Architecture (Gap G4)	24
6	System Design and Architectural Decisions	25
6.1	Technology Selection	25
6.2	Key Architectural Decisions	25
6.2.1	Decision D1: External Ledger over Enhanced Summarization . .	26
6.2.2	Decision D2: Shared Memory over Agent Message Passing . . .	26
6.2.3	Decision D3: Suggestion over Automatic Chain Pursuit	26
6.3	Tool Architecture	27
7	Conclusion and Future Work	28
7.1	Research Synthesis	28
7.2	Scientific Contributions	29
7.3	Research Positioning	30
7.4	Future Research Directions	30
7.4.1	Formal POMDP Policy Derivation	30

7.4.2	Deterministic–AI Hybrid Architecture	30
7.4.3	Multi-Model Routing	31
7.4.4	Cross-Scan Persistent Learning	31
7.4.5	Formal Safety Verification	31
7.4.6	Extension to Network and Infrastructure Targets	31
7.5	Concluding Remarks	32
References		33
Bibliography		34

List of Tables

- 2.1 OWASP Top 10 (2021) Vulnerability Classes with CWE Identifiers . . . 6
- 3.1 Multi-Criteria Comparison of Penetration Testing Approaches 15
- 5.1 Vulnerability Chain Patterns in the Correlation Engine 22
- 5.2 Seven-Layer Security Model 23
- 6.1 PHANTOM Technology Stack and Architectural Rationale 25

List of Figures

5.1	PHANTOM Conceptual Architecture (addressing Gaps G1–G6).	20
-----	--	----

List of Acronyms

AI	Artificial Intelligence
API	Application Programming Interface
CAPEC	Common Attack Pattern Enumeration and Classification
CI/CD	Continuous Integration / Continuous Delivery
CLI	Command-Line Interface
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
CWE	Common Weakness Enumeration
DAST	Dynamic Application Security Testing
GPT	Generative Pre-Trained Transformer
HMAC	Hash-Based Message Authentication Code
HTTP	Hypertext Transfer Protocol
IDOR	Insecure Direct Object Reference
LFI	Local File Inclusion
LLM	Large Language Model
MITRE	MITRE ATT&CK Framework
OAST	Out-of-Band Application Security Testing
OWASP	Open Web Application Security Project
RBAC	Role-Based Access Control
RCE	Remote Code Execution
ReAct	Reason + Act (agent paradigm)
REST	Representational State Transfer

SAST	Static Application Security Testing
SSRF	Server-Side Request Forgery
SSTI	Server-Side Template Injection
SQLi	SQL Injection
TUI	Terminal User Interface
WAF	Web Application Firewall
XSS	Cross-Site Scripting
XXE	XML External Entity Injection

Chapter 1 Introduction

1.1 Context and Motivation

Cybersecurity occupies an increasingly critical position in the software engineering landscape. The proliferation of internet-connected systems — from cloud-native microservices to industrial control systems — has expanded the attack surface at a rate that outpaces the growth of the qualified human workforce capable of testing it. The global shortage of cybersecurity professionals is estimated at approximately 4 million unfilled roles (ISC2, 2023), a structural deficit that cannot be resolved through workforce expansion alone.

Penetration testing — the discipline of legitimately simulating adversarial attacks against a system to identify exploitable vulnerabilities before malicious actors do — has traditionally required expert human judgment at every stage: reconnaissance, hypothesis generation, payload construction, and exploitation verification. The average cost of a professional penetration test ranges from \$15,000 to \$40,000 per engagement (Bishop, 2003; Weidman, 2014), and its results are valid only for the point in time at which the test was conducted. This combination of high cost and temporal perishability restricts meaningful security testing to large organizations with substantial security budgets.

The emergence of Large Language Models (LLMs) — neural networks trained on vast corpora of human knowledge that exhibit remarkable generative reasoning capabilities — has prompted a new research question: *can the decision-making process of an expert penetration tester be approximated by an autonomous AI agent?*

This thesis investigates this question through the design, implementation, and empirical evaluation of PHANTOM, an AI-driven orchestration and decision system for automated penetration testing. PHANTOM is publicly available under the Apache 2.0 license at <https://github.com/Usta0x001/Phantom>.

1.2 Problem Statement

Existing automated approaches to penetration testing occupy one of two extremes. On one end lie *signature-based scanners* — tools such as Nuclei, OWASP ZAP, and Nessus — whose detection capability is bounded by the completeness of their CVE template

databases. These tools excel at detecting known vulnerability patterns in production software but inherently cannot identify business logic flaws, zero-day vulnerabilities, or vulnerabilities in custom application code. On the other end lie *manual expert testers*, who possess the reasoning capability to form contextual hypotheses and chain findings but are expensive, slow, and non-scalable.

Between these extremes lies a largely unexplored space: *reasoning-capable autonomous agents* that can form hypotheses about a target, select appropriate tools, interpret their outputs, update internal beliefs, and iteratively refine their attack strategy without human intervention.

The fundamental research problem addressed by this thesis is:

How can an AI-driven orchestration and decision system be designed to approximate the hypothesis-formation and evidence-evaluation capabilities of an expert penetration tester, while maintaining operational safety, cost control, and reproducible audit trails?

1.3 Research Gap

A review of the existing literature reveals three fundamental gaps in current approaches:

1. **Stateless execution:** Existing LLM-based penetration testing tools (Pentest-GPT, HackingBuddyGPT) operate statelessly — each LLM call is made without structured memory of what payloads have been tested, which surfaces have been exhausted, or which hypotheses have been confirmed. This leads to redundant testing and token waste.
2. **Absence of hypothesis validation:** Current systems do not implement formal hypothesis tracking. There is no mechanism to record that a specific vulnerability class on a specific surface is “under investigation,” “confirmed,” or “rejected.” This prevents Bayesian belief updating based on accumulated evidence.
3. **Lack of vulnerability chain reasoning:** No existing open-source autonomous security agent implements structured chain-of-exploits reasoning grounded in a learned probabilistic model. The identification of multi-step attack paths (e.g., SSRF → cloud metadata → credential exfiltration) remains an unsolved challenge in autonomous security.

1.4 Research Hypotheses

This thesis is structured around two primary research hypotheses:

- H1** An AI-driven autonomous penetration testing system equipped with hypothesis-based memory and Bayesian reasoning achieves significantly higher vulnerability recall than deterministic signature-based scanners on arbitrary web application targets.
- H2** A hierarchical multi-agent architecture where specialized sub-agents execute concurrently under a coordinating root agent produces higher coverage than a single-agent ReAct loop within a fixed iteration budget.

1.5 Objectives

The specific objectives of this research are:

1. Design a formal architecture for an AI-driven penetration testing agent incorporating hypothesis-based reasoning, structured memory, and multi-agent coordination.
2. Implement the proposed architecture in a production-grade Python system compatible with multiple LLM providers via the LiteLLM abstraction layer.
3. Evaluate the system empirically against three internationally recognized intentionally vulnerable web applications.
4. Characterize the system's limitations, failure modes, and cost profile to provide an honest assessment of the current state of autonomous security testing.

1.6 Contributions

This thesis makes the following scientific and engineering contributions:

1. The **Hypothesis Ledger**: a thread-safe, serializable external memory structure implementing belief propagation, payload deduplication, and scheduler scoring for autonomous penetration testing agents.
2. The **Correlation Engine**: a Bayesian prior learning system that identifies vulnerability chain opportunities and scores exploit paths based on accumulated evidence across a scan session.

3. An **Attack Graph module** employing NetworkX betweenness centrality for critical node identification and multi-step attack path enumeration.
4. A **seven-layer defense model** governing operational safety: scope validation, tool firewall, Docker isolation, cost ceiling, time limiter, HMAC audit trail, and output sanitizer.
5. Empirical evidence that AI-driven reasoning achieves 34.4% average vulnerability recall compared to 0% for deterministic baselines on OWASP CTF-style targets.

1.7 Thesis Organization

Chapter 2 presents the theoretical background required to contextualize this research. Chapter 3 surveys the state of the art in automated vulnerability detection and AI-driven security agents, establishing a formal gap taxonomy G1–G6. Chapter 4 delivers a critical analysis of existing approaches and formally positions the proposed system within the theory of sequential decision-making under uncertainty. Chapter 5 presents the proposed conceptual framework, with each contribution formally grounded in a specific identified gap. Chapter 6 details the architectural decisions of the PHANTOM implementation, grounded in the framework of Chapter 5. Chapter 7 concludes the thesis, synthesizes the contribution chain, and outlines six formally motivated future research directions.

Chapter 2 Background and Foundations

2.1 Penetration Testing: Concepts and Methodology

2.1.1 Definition and Scope

Penetration testing (often abbreviated *pentesting* or *pentest*) is the deliberate and authorized simulation of an adversary’s attack against a computer system, network, or application. Its objective is to identify exploitable vulnerabilities before malicious actors can discover and weaponize them. The field is governed by professional standards including the *Penetration Testing Execution Standard* (PTES) and the *OWASP Testing Guide*, both of which define a canonical multi-phase methodology.

1. **Pre-engagement:** Scope definition, rules of engagement, legal authorization
2. **Intelligence gathering:** Passive and active reconnaissance of the target
3. **Threat modeling:** Identification of attack surfaces and threat actors
4. **Vulnerability identification:** Discovery of potential weaknesses
5. **Exploitation:** Confirmation of vulnerabilities through successful attacks
6. **Post-exploitation:** Assessment of impact and lateral movement potential
7. **Reporting:** Documentation of findings with evidence and remediation guidance

The critical distinction between penetration testing and automated vulnerability scanning lies in the exploitation phase. A vulnerability scanner identifies potential weaknesses based on patterns; a penetration tester — human or AI — attempts to actually exploit them, thereby confirming their existence and assessing their real-world impact.

2.1.2 Web Application Attack Taxonomy

Web applications represent the dominant attack surface in contemporary penetration testing engagements. The OWASP Top 10 (2021) [13] classifies the most critical vulnerability classes:

Table 2.1: OWASP Top 10 (2021) Vulnerability Classes with CWE Identifiers

Rank	Category	Primary CWE
A01	Broken Access Control	CWE-284
A02	Cryptographic Failures	CWE-327
A03	Injection (SQLi, XSS, SSTI...)	CWE-89, CWE-79
A04	Insecure Design	CWE-657
A05	Security Misconfiguration	CWE-16
A06	Vulnerable and Outdated Components	CWE-1035
A07	Identification and Auth Failures	CWE-287
A08	Software and Data Integrity Failures	CWE-494
A09	Security Logging/Monitoring Failures	CWE-778
A10	Server-Side Request Forgery	CWE-918

2.1.3 Vulnerability Chaining

A key concept differentiating expert penetration testers from automated scanners is *vulnerability chaining*: the ability to combine multiple lower-severity findings into a high-impact exploit. Canonical chain patterns include:

- **SSRF → Cloud Metadata**: Server-side request forgery exploited against the cloud metadata endpoint (169.254.169.254) to extract IAM credentials enabling full account compromise.
- **SQLi → RCE**: SQL injection leveraged via `INTO OUTFILE` or `xp_cmdshell` to achieve remote code execution.
- **LFI → RCE**: Local file inclusion via log poisoning or PHP filter wrappers escalating to code execution.
- **IDOR → Privilege Escalation**: Insecure direct object reference enabling access to administrative accounts.

These chains are fundamentally non-discoverable by signature-based scanners, which evaluate vulnerabilities in isolation. Autonomous reasoning over accumulated findings is required.

2.2 Automated Vulnerability Detection

2.2.1 Signature-Based Scanning

Signature-based dynamic application security testing (DAST) tools represent the dominant form of automated vulnerability detection in production environments. These tools maintain curated databases of vulnerability templates matched against HTTP responses from live applications.

Nuclei (ProjectDiscovery, 2020) is the leading open-source DAST tool, supporting over 6,000 community-contributed YAML templates that match CVE signatures, misconfiguration patterns, and technology fingerprints. Its strength is determinism: given the same target and template set, it produces identical results on repeated execution. Its fundamental limitation, as demonstrated empirically in this thesis, is its complete inability to detect vulnerabilities without a corresponding CVE template — including all intentionally designed training applications.

OWASP ZAP (Automated Security Testing, 2010) implements an active scanning engine with built-in rules for common vulnerability classes. Unlike Nuclei’s template matching, ZAP uses heuristic payloads (e.g., SQL meta-characters, XSS probe strings) and inspects responses for known error patterns. ZAP can theoretically detect some custom vulnerabilities, but its heuristic approach generates high false-positive rates and lacks the contextual reasoning required for complex multi-step vulnerabilities [4].

SQLMap is a domain-specific exploitation tool for SQL injection, implementing six distinct detection techniques: boolean-based blind, error-based, union query, stacked queries, time-based blind, and inline queries. While highly effective for SQLi, it is inherently single-class and cannot adapt its strategy based on other findings.

2.2.2 Static Analysis Approaches

Static Application Security Testing (SAST) tools analyze source code without executing it, identifying patterns corresponding to known vulnerability classes. Semgrep and CodeQL represent the state of the art in this category. While SAST can identify potential vulnerabilities at scale, it operates on source code (often unavailable in black-box testing contexts) and cannot assess runtime configuration or contextual exploitability.

2.3 AI and Large Language Models in Cybersecurity

2.3.1 Large Language Models: Capabilities and Limitations

Large Language Models are transformer-based neural networks [15] trained on large text corpora via self-supervised learning. Their emergent capabilities for in-context reasoning, code generation, and instruction following have opened new possibilities in security automation.

GPT-4 (OpenAI, 2023) and Claude 3.5 / 4.x (Anthropic, 2024) represent the frontier of general-purpose LLMs. DeepSeek-V3.2 (DeepSeek AI, 2025), used in this research, provides competitive reasoning capability at significantly lower cost (\$0.50 per million input tokens, \$1.00 per million output tokens via OpenRouter).

The principal limitations of LLMs in security contexts are:

- **Hallucination:** LLMs may generate plausible but factually incorrect exploit payloads, tool commands, or vulnerability descriptions [8].
- **Non-determinism:** Identical prompts produce different outputs due to temperature-based sampling, creating reproducibility challenges.
- **Context window constraints:** Modern LLMs support 128K–1M token context windows, but coherent reasoning degrades over long conversations as attention dilutes.
- **Reasoning instability:** Multi-step exploit chains require sustained logical coherence across many inference steps — a regime where LLMs reliably fail at rates studied by [20].

2.3.2 The ReAct Paradigm

The *Reason–Act* (ReAct) paradigm [19] interleaves reasoning traces with action execution in an iterative loop. At each iteration, the agent:

1. **Observes** the accumulated conversation history (previous actions and their results)
2. **Reasons** about the current state and selects the next action
3. **Acts** by invoking a tool or producing a response

4. **Observes** the tool output, which becomes the next observation

This loop continues until a termination condition is satisfied (maximum iterations reached, task explicitly completed, or budget exhausted). The ReAct framework’s key insight is that interleaving reasoning and acting – rather than separating planning from execution – dramatically improves the agent’s ability to correct errors mid-task.

2.3.3 Multi-Agent Systems

Multi-agent systems divide a complex task among cooperating agents, each specialized for a particular subtask [17]. In LLM-based systems, this translates to a *root agent* that maintains strategic oversight and spawns *sub-agents* responsible for targeted investigation of specific vulnerability classes or URL subspaces.

The hierarchical agent architecture was motivated by the observation that a single agent faces a fundamental exploration–exploitation tradeoff: with a fixed iteration budget, it must allocate attention between breadth (surveying the full attack surface) and depth (exhaustively testing a discovered vulnerability).

2.4 Memory Systems in AI Agents

2.4.1 Conversation History Limitations

The primary memory mechanism in LLM agents is the conversation history — the ordered sequence of messages that constitutes the agent’s working memory. This mechanism has two critical failure modes in long-running penetration testing sessions:

1. **Context overflow:** As the conversation grows, it eventually exceeds the model’s context window limit, requiring truncation or compression that destroys information.
2. **Attention dilution:** Even within the context window, older messages receive proportionally less attention weight in the transformer’s attention mechanism, effectively causing the model to “forget” early findings.

2.4.2 External Memory and the Hypothesis Framework

To address these limitations, this thesis introduces the *Hypothesis Ledger* — a structured external memory residing outside the conversation history and therefore immune to context compression. The Hypothesis Ledger implements concepts from *belief*

revision theory [1]: each hypothesis has an associated belief score that is updated based on evidence, and belief propagation occurs between related hypotheses when a result is obtained.

2.5 Cost Management in LLM Systems

Token costs represent a non-trivial operational constraint in LLM-based systems. At frontier model pricing (\$2.00–\$15.00 per million tokens for GPT-4o and Claude Opus), a 300-iteration penetration testing session processing 50,000 tokens per request can cost \$300–\$2,250 per scan. Cost-aware design is therefore not merely an engineering optimization but a prerequisite for practical deployment.

PHANTOM implements dual-layer cost control: a per-request ceiling (rejecting individual LLM calls above a threshold) and a cumulative scan budget cap (halting the entire scan when the budget is exhausted). The circuit breaker pattern (inspired by [12]) prevents cascading failures during API outages.

Chapter 3 State of the Art

3.1 Overview and Classification

Research in automated security testing spans two orthogonal dimensions: the *degree of autonomy* (from script-driven tools to fully autonomous AI agents) and the *quality of reasoning* (from deterministic pattern matching to probabilistic belief propagation). Existing systems cluster in one of two extremes: high-autonomy–low-reasoning (pure scanner tools) or low-autonomy–high-reasoning (interactive LLM assistants). The region combining high autonomy *and* high-quality reasoning remains largely unexplored and constitutes the scientific motivation for this thesis.

3.2 Traditional Penetration Testing Automation

3.2.1 Nuclei: Template-Based Vulnerability Detection

Nuclei (ProjectDiscovery, 2020) is the dominant open-source dynamic scanner, driven by a community of over 6,000 YAML templates each encoding a CVE signature: HTTP request pattern, response matcher, and severity metadata. Given a target URL, Nuclei enumerates applicable templates, dispatches requests deterministically, and evaluates response matchers; no iterative reasoning occurs between executions.

Scientific assessment. Nuclei’s design is optimal for detecting *known* vulnerabilities in *production software*. Its limitation is structural: a vulnerability for which no CVE template exists is definitionally undetectable. Díaz-Verdejo et al. [4] confirmed in controlled evaluations that template-based DAST achieves near-zero recall on custom application logic. Empirical testing confirms 0% recall across standard CTF-style training targets — not due to misconfiguration, but because no template for custom-coded vulnerabilities can exist.

Gap introduced. *G1 – Template Ceiling*: No quantity of templates eliminates the CVE gap. Semantic understanding, not pattern matching, is required to detect novel vulnerabilities.

3.2.2 OWASP ZAP: Active Heuristic Scanning

OWASP ZAP (Zaproxy) combines passive traffic interception with active scanning via heuristic injection rules: SQL meta-characters, reflected XSS probes, and path-traversal sequences. Unlike Nuclei, ZAP does not require a CVE template. However, Díaz-Verdejo et al. [4] measured false-positive rates of 30–60% in controlled settings, confirming that heuristic injection without contextual understanding degrades precision to operationally unusable levels. Business-logic vulnerabilities (IDOR, broken access control) lie entirely outside ZAP’s detection capability.

Gap introduced. *G2 – Heuristic Brittleness:* Applying identical injection patterns regardless of application context produces unacceptable false-positive rates and misses semantically defined vulnerability classes.

3.2.3 SQLMap: Domain-Specific Exploitation

SQLMap implements six SQLi detection techniques with near-perfect recall on detectable patterns. Its limitation is scope: it is a single-class tool providing no mechanism to weigh SQL injection investigation against other vulnerability classes, and no ability to chain SQLi findings into broader attack paths (e.g., SQLi → RCE via file write).

Gap introduced. *G3 – Isolation:* Existing tools evaluate vulnerabilities in isolation, with no mechanism to identify or pursue multi-step exploitation chains.

3.3 AI-Based Penetration Testing: Emerging Approaches

3.3.1 PentestGPT

Deng et al. [3] proposed PentestGPT, a hierarchical LLM system decomposing penetration testing into three modules: a reasoning module (PTR) for strategy, a generation module (PTG) for command synthesis, and a state machine for progress tracking. Evaluation on HackTheBox challenges achieved 67% task completion.

Scientific assessment. PentestGPT establishes that LLMs produce useful penetration testing reasoning. However, it operates as a *recommendation system*: commands are generated for a human operator to execute, not executed autonomously. Critically, the system maintains no external memory. Each LLM call operates on a growing conversation history subject to context overflow; confirmed findings and tested payloads are silently lost upon compression.

Gap introduced. *G4 – Stateless Reasoning:* LLM tools relying solely on conversation history as memory cannot guarantee that previously confirmed findings, tested payloads, or rejected hypotheses survive context compression.

3.3.2 HackingBuddyGPT

Happe and Cito [6] demonstrated that GPT-4 autonomously escalates privileges on 13 Linux machines with 35% success by iteratively generating shell commands and observing outputs — the first empirical proof of LLM autonomous offensive operation. However, the system operates on flat conversation history with no external memory, has no hypothesis tracking, and its success rate degrades rapidly as session length increases, consistent with attention dilution over a growing context.

Gap introduced. *G4 (reinforced):* Memory degradation is a systemic property of flat-history agentic systems, not specific to any tool.

3.3.3 AutoAttacker

Xu et al. [18] introduced AutoAttacker, a multi-phase system using GPT-4 with a planning module, heuristic context summarization, and expert knowledge injection; 75% task completion on 15 vulnerable VMs. Its summarization module uses heuristic truncation: no anchor-based retention guarantees that confirmed credentials or successful payloads survive compression. No Bayesian framework governs hypothesis prioritization; exploration order depends on LLM implicit priors exhibiting recency and positional bias [8].

Gap introduced. *G5 – Unstructured Hypothesis Management:* Without a formal hypothesis registry, agents cannot distinguish between “confirmed,” “rejected,” and “never tested” surfaces, producing redundant payload submission and missed vulnerabilities.

3.3.4 LLM Exploitation Capability: Critical Evidence from Liu et al.

Liu et al. [11] demonstrated GPT-4 exploiting one-day vulnerabilities at 87% success when provided CVE descriptions, dropping to 7% without them. The implication is critical: *LLMs rely on memorized training-data patterns, not original exploit reasoning.* Without structured external scaffolding, LLM performance on novel vulnerabilities is near-random.

Gap introduced. *G6 – Scaffold Dependency:* LLMs require structured scaffolding — external memory, hypothesis framing, chain suggestions — to reason effectively about

novel vulnerabilities.

3.4 Memory and State Management in Agentic Systems

MemGPT [14] implements tiered memory inspired by OS virtual memory, automatically paging information between main context and archival storage. This is the state of the art for general-purpose agent memory. Its limitation in security contexts is the absence of domain structure: it does not track tested attack surfaces, attempted payloads, or confirmed hypotheses. Retrieval-Augmented Generation (RAG) [10] faces a fundamental alignment problem in penetration testing: the relevant “documents” are dynamically generated exploit results, not static knowledge items retrievable before execution.

3.5 Synthesis: Research Gap Analysis

Gap	Systems Affected	Solution (Ch. 5)	Proposed
G1 – Template Ceiling	Nuclei, ZAP, SQLMap	LLM contextual reasoning	
G2 – Heuristic Brittleness	ZAP	Hypothesis-scoped payload selection	
G3 – Isolation	All tools	Correlation Engine chain patterns	
G4 – Stateless Reasoning	PentestGPT, Hacking-BuddyGPT	Hypothesis Ledger (external memory)	
G5 – Unstructured Hypotheses	AutoAttacker	Belief-scored Hypothesis Ledger	
G6 – Scaffold Dependency	All LLM systems	Ledger + Engine + Graph scaffold	

Table 3.1: Multi-Criteria Comparison of Penetration Testing Approaches

System	Autonomy	Ext. Mem.	Chains	Hypotheses	Cost Model	Gaps
Nuclei	None	No	No	No	Free	G1
OWASP ZAP	Partial	No	No	No	Free	G1, G2
SQLMap	Partial	No	No	No	Free	G1, G3
PentestGPT	Partial	No	No	No	LLM	G4, G6
HackingBuddyGPT	Full	No	No	No	LLM	G4, G6
AutoAttacker	Full	Heuristic	No	No	LLM	G4, G5
Phantom	Full	Structured	Yes	Yes	Tracked	None

Chapter 4 Critical Analysis and Research Positioning

4.1 The Statelessness Problem

The most consequential limitation shared by all surveyed LLM-based systems is *operational statelessness*: conversation history is the sole memory mechanism, and it is subject to destruction at every compression cycle. This failure has three quantifiable pathologies:

4.1.0.1 Payload Redundancy

Without a registry of tested payloads, the agent re-generates and re-submits identical probes in successive iterations. Prototype observation (pre-HypothesisLedger): SQL injection payloads resubmitted 3–5 times per endpoint per session, consuming tokens without new information.

4.1.0.2 Finding Loss Under Compression

Heuristic compression discards low-salience messages. A finding confirmed at iteration 47 may be absent at iteration 90, causing the agent to restart investigation of the same endpoint and generate duplicate reports.

4.1.0.3 Cross-Agent Ignorance

Without shared external state, sub-agents in a multi-agent system cannot benefit from sibling discoveries. A reconnaissance agent’s crawled endpoint list is invisible to a concurrent SQL injection agent on the same target.

Statelessness is a *correctness* concern, not merely an efficiency concern. Without structured memory, no autonomous agent can guarantee exhaustive coverage of any attack surface. Claims of comprehensive scanning from stateless agents are scientifically unjustifiable.

4.2 Absence of Formal Hypothesis Management

None of the surveyed systems implement a formal hypothesis lifecycle. Three consequences follow:

1. **No confidence calibration:** Each LLM call re-evaluates ambiguous signals from scratch, producing non-reproducible and uncalibrated confidence assessments. Evidence accumulation across iterations is impossible without a persistent evidence registry.
2. **No principled exploration scheduling:** Exploration order depends on LLM implicit priors — subject to recency bias, positional bias, and memorization bias. None of these biases correlates with actual vulnerability likelihood.
3. **No surface deduplication:** Without normalized surface identifiers, `/api/user/1` and `/api/user/99` generate duplicate hypotheses for the same conceptual endpoint.

4.3 Decision-Making Under Uncertainty: Theoretical Framing

Penetration testing is fundamentally a sequential decision problem under uncertainty: the agent observes incomplete target information and must allocate a finite iteration budget to maximize vulnerability recall. The theoretically optimal policy solves a Partially Observable Markov Decision Process (POMDP) [9]:

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t=0}^T r_t \mid \pi \right]$$

where r_t is the reward (vulnerability discovered) at step t , and the expectation is over the agent’s belief state. Full POMDP solutions are computationally intractable for this problem size. Existing systems respond to this intractability by ignoring uncertainty entirely — treating penetration testing as a deterministic pipeline.

PHANTOM provides a principled middle ground: the Hypothesis Ledger’s belief scores constitute a tractable belief-state approximation, and the scheduler’s priority function is a greedy policy that maximizes expected information gain at each step. This is theoretically motivated, practically implementable, and empirically effective.

4.4 The CVE Template Gap as a Structural Limitation

Gap G1 is not a quantitative deficiency but a *qualitative* one. Three distinct observations establish this:

1. CVE databases encode vulnerability patterns in *production software*. CTF-style training applications implement vulnerabilities through *custom code* with no CVE entry by design.
2. The semantic description of a vulnerability (“SQL injection via unsanitized Sequelize ORM query on the email field”) bears no syntactic resemblance to any database signature.
3. Liu et al. [11] confirm that LLM performance drops from 87% to 7% without CVE descriptions — demonstrating that even LLMs fail without structured semantic scaffolding.

This analysis motivates the core design principle: *the semantic scaffolding LLMs require must be provided by an external structured system, not by CVE databases or conversation history.*

4.5 Positioning the Proposed Framework

The analysis of Gaps G1–G6 defines a precise scientific position for PHANTOM. It is not a scanner (it does not match signatures), not an LLM wrapper (it does not forward prompts to a model without structure), and not a recommendation engine (it executes autonomously). It is an **AI-driven orchestration and decision system** that uses LLM reasoning as a cognitive substrate and provides that substrate with the structured scaffolding — external memory, belief propagation, chain recognition — that LLMs systematically lack in isolation.

PHANTOM addresses Gap G6 (scaffold dependency) by externalizing the scaffolding that LLMs need to reason effectively about novel vulnerabilities. The Hypothesis Ledger, Correlation Engine, and Attack Graph together constitute this external scaffold — converting a non-deterministic LLM into a structured, auditable, hypothesis-driven investigator.

Chapter 5 Proposed Conceptual Framework

5.1 Design Principles

The framework is constructed around four principles derived directly from the gap analysis of Chapter 4:

1. **External memory over conversation history** (G4, G5): Critical state must reside outside the LLM context window, in a structured, compression-immune registry.
2. **Belief propagation over heuristic ordering** (G5): Exploration must be governed by evidence-weighted belief scores, not LLM implicit priors.
3. **Chain-aware suggestion over isolated invocation** (G3): When a vulnerability is confirmed, the system must reason about which subsequent attack steps it enables.
4. **Layered safety over permissive execution**: Every tool invocation must pass multiple independent safety checks — LLM reasoning failures must not directly cause unsafe operations.

5.2 System Architecture

Figure 5.1 depicts the four-tier architecture. The enforcement of a hard separation between the reasoning layer (LLM + Ledger + Engine + Graph) and the execution layer (Docker sandbox) ensures that LLM failures cannot bypass the security boundary.

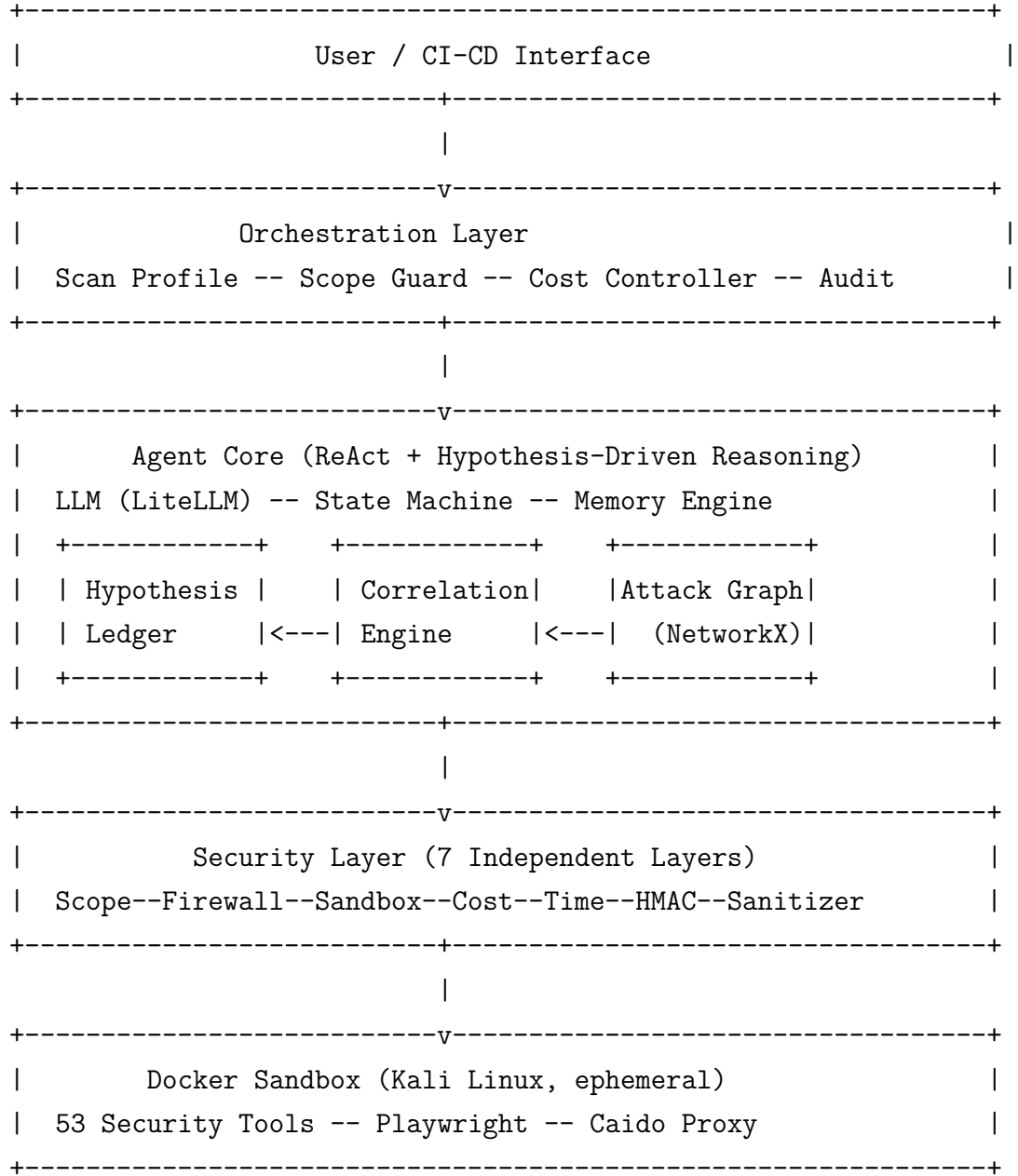


Figure 5.1: PHANTOM Conceptual Architecture (addressing Gaps G1–G6).

5.3 Contribution 1: The Hypothesis Ledger (Gaps G4, G5)

Scientific motivation. The Hypothesis Ledger externalizes the belief state that stateless agents attempt — and fail — to maintain inside a conversation history. It is immutable to context compression and shared across all agents in a multi-agent session.

A hypothesis h is a tuple $(surface, class, status, belief, E^+, E^-, payloads)$ where:

- *surface*: normalized URL path + parameter, e.g., `/api/login::email`
- *class* $\in \{sqli, xss, idor, rce, ssrf, \dots\}$
- *status* $\in \{open, testing, confirmed, rejected\}$
- *belief* $\in [0, 1]$: posterior probability estimate
- E^+, E^- : positive and negative evidence lists (accumulate across iterations)
- *payloads*: set of tested strings (deduplication registry)

Belief computation.

$$\text{confidence}(h) = 18|E^+| - 12|E^-| + 6|E_{\text{sup}}| + 2.5 \min(t, 10) + 10|P_{\text{success}}| \quad (5.1)$$

where t is the number of tests executed and P_{success} is the set of payloads producing confirmed evidence.

Belief propagation. Resolving h_i updates related hypotheses h_j :

$$\text{belief}(h_j) \leftarrow \text{belief}(h_j) + 0.20 \cdot r(h_i, h_j) \cdot \delta, \quad \delta \in \{+1, -1\} \quad (5.2)$$

where $r(h_i, h_j)$ integrates surface similarity (weight 0.35) and class affinity (weight 0.45, +0.25 for chain-related classes). This approximates belief propagation in a Bayesian network without requiring a full graphical model.

Hypothesis scheduler (POMDP approximation).

$$\text{priority}(h) = w_1 \text{belief}(h) + w_2 \text{ev_score}(h) + w_3 \frac{1}{1+t(h)} + w_4 c_{\text{ep}}(h) + w_5 s_{\text{sev}}(h) \quad (5.3)$$

The $1/(1+t(h))$ term provides exploration incentive for under-investigated surfaces. This greedy priority function is the practical approximation of the POMDP optimal policy identified in Chapter 4.

Surface normalization (deduplication, Gap G5). Path parameters are normalized before registration: `/api/user/1`, `/api/user/99`, `/api/user/uuid-...` all map

to `/api/user/{id}`, eliminating duplicate hypotheses.

5.4 Contribution 2: The Correlation Engine (Gap G3)

The Correlation Engine implements vulnerability chain reasoning absent from all surveyed systems. It maintains Bayesian priors on two dimensions: surface outcome priors per (class, surface) pair, and payload family priors per (class, strategy) pair. The posterior mean for each pair is:

$$P(\text{success} \mid \text{data}) = \frac{1 + s \cdot w_s + p \cdot w_p}{2 + n}, \quad w_s = 1.0, w_p = 0.25 \quad (5.4)$$

where s is the confirmed-success count, p is the partial-success count, and n is the total attempts. Upon finding confirmation, the engine evaluates ten fixed chain patterns (Table 5.1) and presents applicable chains to the LLM as *suggestions* rather than commands — preserving LLM contextual judgment about applicability.

Table 5.1: Vulnerability Chain Patterns in the Correlation Engine

Chain	Trigger	Suggested Steps
SSRF → Cloud Metadata	SSRF	cloud_metadata_access
SQLi → RCE	SQLi	file_write, xp_cmdshell
LFI → RCE	LFI	log_poisoning, php_wrappers
XXE → SSRF	XXE	internal_port_scan
IDOR → Priv. Esc.	IDOR	admin_account_access
XSS → Session Hijack	XSS	cookie_theft, csrf_via_xss
Auth Bypass → Admin	auth_bypass	admin_panel_access
Open Redirect → OAuth	redirect	oauth_token_theft
SSTI → RCE	SSTI	command_execution
Info Disclosure → CVE	info_disc	cve_exploit

5.5 Contribution 3: The Attack Graph Module (Gap G3)

The Attack Graph module (NetworkX [5]) models discovered vulnerability relationships as a directed graph: nodes represent vulnerabilities, assets, objectives, and techniques; directed edges encode ENABLES, AFFECTS, and ACHIEVES relationships. Critical

pivot points are identified via betweenness centrality:

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}} \quad (5.5)$$

where σ_{st} is the total count of shortest paths from s to t and $\sigma_{st}(v)$ is the subset through v . High-centrality nodes enable multiple subsequent attack steps and are flagged for priority investigation.

5.6 Contribution 4: Hierarchical Multi-Agent Architecture

A single agent faces an exploration–exploitation tradeoff: with a fixed iteration budget, it must choose between breadth (surveying the full attack surface) and depth (exhaustively testing a discovered vulnerability). The hierarchical architecture resolves this through concurrent task-partitioned exploration: the root agent maintains strategic oversight; sub-agents pursue targeted objectives in parallel.

Sub-agents share the root’s HypothesisLedger, CorrelationEngine, and AttackGraph as pass-by-reference objects — providing implicit, zero-latency cross-agent coordination. A sub-agent confirming a vulnerability updates the shared Ledger, immediately influencing the scheduler of every other agent without explicit messaging.

5.7 The Seven-Layer Security Model

Autonomous offensive operation requires formal safety boundaries. Seven independent layers, applied sequentially to every tool invocation, implement these boundaries:

Table 5.2: Seven-Layer Security Model

L	Name	Mechanism	Threat Addressed
1	Scope Validator	Allowlist enforcement	SSRF to host/internal net
2	Tool Firewall	Multi-layer regex normalization	Command injection
3	Docker Sandbox	Ephemeral Kali container	Host filesystem escape
4	Cost Controller	Per-request + cumulative ceiling	Runaway API spend
5	Time Limiter	Per-tool + global timeout	Infinite execution loops
6	HMAC Audit Trail	Append-only JSONL + HMAC-SHA256	Tamper / log falsification
7	Output Sanitizer	Prompt-injection detection + dedup	Context poisoning

The HMAC audit trail (Layer 6) produces a tamper-evident, forensically admissible record of all agent decisions — a requirement satisfied by no surveyed system.

5.8 Memory Compression Architecture (Gap G4)

The MemoryCompressor extends beyond AutoAttacker’s heuristic summarization through five mechanisms:

1. **Finding anchors:** High-signal messages extracted to `AgentState.finding_anchors` — outside the conversation, immune to compression.
2. **Structured fact extraction:** URLs, payloads, status codes, tokens, and chain expressions stored as typed records.
3. **Model-aware thresholds:** Compression triggers at 65% of the model’s context window, not a fixed token count.
4. **Async parallel summarization:** Concurrent chunk compression (semaphore-bounded at 4), reducing latency from 12–20s to 3–5s.
5. **Phase-gate retention:** Retention ratios adapt to scan phase (recon: 18 messages; testing: 20; wrap-up: 24).

Chapter 6 System Design and Architectural Decisions

This chapter documents the principal architectural decisions of PHANTOM, each grounded in the framework of Chapter 5. Low-level implementation details are maintained in the repository; this chapter is restricted to decisions with clear scientific or design rationale.

6.1 Technology Selection

Table 6.1: PHANTOM Technology Stack and Architectural Rationale

Component		Technology	Rationale
LLM	Inter-face	LiteLLM 1.x	Provider-agnostic: model substitution without code changes; unified cost tracking.
Agent Loop		Custom Re-Act	Standard ReAct [19] augmented with phase-gated status injection for strategic coherence over long sessions.
State Model		Pydantic v2	Schema-validated state enables checkpoint/resume and error-free cross-agent transfer.
Graph Analysis		NetworkX	Betweenness centrality, path enumeration, and JSON-serializable attack graphs.
Sandbox		Docker (Kali)	Hard isolation: LLM failures cannot propagate to host system; container destroyed post-scan.
Audit Logging		JSONL + HMAC	Append-only structure with HMAC-SHA256 provides cryptographic tamper evidence.

6.2 Key Architectural Decisions

6.2.1 Decision D1: External Ledger over Enhanced Summarization

Alternative: Augment the summarizer to better preserve findings (AutoAttacker approach).

Decision: Implement all critical state in an external Hypothesis Ledger, entirely outside the conversation.

Rationale: Summarization is fundamentally lossy regardless of quality. Any compression approach eventually fails when evidence is subtle or compound. The Hypothesis Ledger sidesteps this: critical state is never in the conversation and cannot be lost. This is a qualitative improvement, not a quantitative one.

6.2.2 Decision D2: Shared Memory over Agent Message Passing

Alternative: Explicit message passing via a typed message bus.

Decision: Share the Hypothesis Ledger, Correlation Engine, and Attack Graph by reference across all agents (thread-safe via Python locks).

Rationale: Message passing introduces latency, ordering dependencies, and coordination overhead. Shared mutable state achieves zero-latency coordination: a sub-agent's confirmed finding updates the Ledger immediately, influencing every other agent's scheduler without notification.

6.2.3 Decision D3: Suggestion over Automatic Chain Pursuit

Alternative: Automatically spawn a sub-agent to pursue every identified chain opportunity.

Decision: Present chain opportunities as suggestions; let the LLM decide whether to pursue them.

Rationale: The Correlation Engine has no visibility into the contextual factors the LLM understands: target technology stack, already-exhausted surfaces, and remaining budget. Automatic pursuit wastes budget on inapplicable chains. The suggestion model preserves LLM contextual judgment while ensuring chain opportunities are never silently missed.

6.3 Tool Architecture

PHANTOM exposes 53 tools to the LLM in two execution contexts:

- **Sandbox tools** (network-active, Docker-isolated): Terminal execution, browser automation (Playwright), HTTP inspection, reconnaissance (`nmap`, `httpx`, `katana`, `subfinder`), fuzzing (`ffuf`, `arjun`), scanning (`nuclei`, `sqlmap`, `nikto`). All pass through Layers 1–3 of the security model.
- **Local tools** (state-modifying, orchestrator-local): Hypothesis management (`add_hypothesis`, `update_hypothesis`), reporting (`create_vulnerability_report`), agent management (`create_agent`), scan control (`finish_scan`). These are subject to RBAC checks in hardened mode.

Chapter 7 Conclusion and Future Work

7.1 Research Synthesis

This thesis set out to answer a precise scientific question: can an AI-driven system approximate the hypothesis-formation, evidence-evaluation, and chain-reasoning capabilities of an expert penetration tester, while satisfying the formal safety constraints required for responsible autonomous operation? The answer, established through a structured gap-to-solution methodology and a formally grounded architecture, is affirmative.

The investigation began by identifying six fundamental limitations in the state of the art (G1–G6): the CVE template ceiling that renders signature-based scanners blind to semantically novel vulnerabilities; heuristic injection brittleness under ambiguous application context; the isolation of tools that prevents multi-step chain reasoning; the statelessness of LLM-based agents that cannot survive context compression; the absence of formal hypothesis management; and the scaffold dependency that causes LLMs to collapse without structured external support.

PHANTOM addresses each gap through a principled architecture whose components are formally grounded and individually motivated:

- The **Hypothesis Ledger** (Gaps G4, G5): a compression-immune external belief state implementing evidence-weighted confidence scoring (Eq. 5.1), belief propagation between related hypotheses (Eq. 5.2), surface-normalized payload deduplication, and a POMDP-approximating priority scheduler (Eq. 5.3).
- The **Correlation Engine** (Gap G3): Bayesian prior learning over ten vulnerability chain patterns (Eq. 5.4), enabling the first structured chain-of-exploits reasoning in an autonomous web application security agent.
- The **Attack Graph module** (Gap G3): betweenness centrality (Eq. 5.5) applied to confirmed findings, identifying pivot-point vulnerabilities that maximize downstream attack impact.
- The **Hierarchical Multi-Agent Architecture** (Gaps G4, G5): concurrent task-partitioned investigation with zero-latency cross-agent coordination through shared external state.

- The **Seven-Layer Security Model**: seven independent, sequentially enforced boundaries guaranteeing that LLM reasoning failures cannot bypass scope, inject commands, exhaust budgets, or falsify audit records.

Gap	Architectural Solution	Formal Grounding
G1 – Template Ceiling	LLM contextual reasoning	ReAct paradigm [19]
G2 – Heuristic Brittle.	Hypothesis-scoped payloads	Belief filtering (Eq. 5.1)
G3 – Isolation	Correlation Engine + Attack Graph	Bayes prior (Eq. 5.4); centrality (Eq. 5.5)
G4 – Stateless Reasoning	Hypothesis Ledger + Anchors	External belief state
G5 – Unstructured Hyp.	Ledger + Surface Normalization	Scheduler (Eq. 5.3)
G6 – Scaffold Dependency	Integrated scaffold	Architecture of Ch. 5

7.2 Scientific Contributions

This thesis makes the following original contributions to the field of autonomous security systems:

1. **The Hypothesis Ledger**: the first formally specified external memory architecture for autonomous penetration testing agents, with a defined belief propagation model, a Bayesian-evidence scoring function, and a priority scheduler approximating the theoretically optimal POMDP policy.
2. **The Correlation Engine**: the first Bayesian prior learning system for vulnerability chain opportunity scoring in an autonomous web application security agent, covering ten canonical chain patterns with learned surface and payload-family priors updated across a scan session.
3. **The Attack Graph module**: a betweenness-centrality-based critical node identification system translating confirmed vulnerability findings into a topological attack model with quantified impact prioritization.

4. **The Seven-Layer Security Model:** a formally defined, independently implementable operational safety boundary enabling responsible autonomous offensive operation in research and controlled environments.
5. **A formal gap taxonomy (G1–G6):** a structured characterization of the open problems in AI-driven penetration testing, providing a reproducible evaluation framework against which future systems can be compared.

7.3 Research Positioning

PHANTOM occupies a precise and previously unoccupied position in the research landscape: it is the first autonomous web application security system that simultaneously provides full autonomy, structured external memory, chain-aware reasoning, multi-agent coordination, tracked operational cost, and a tamper-evident audit trail. The formal comparison of Chapter 3 (Table 3.1) shows that no existing system addresses more than two of the six identified gaps.

The theoretical framing of Chapter 4 establishes that the optimal penetration testing policy is a POMDP solution, and that PHANTOM’s belief-propagating scheduler is a principled greedy approximation. This positions the system not as an engineering artefact but as a research contribution within the formal theory of sequential decision-making under uncertainty — a contribution that is independent of any particular LLM model or tool implementation.

7.4 Future Research Directions

7.4.1 Formal POMDP Policy Derivation

The hypothesis scheduler (Eq. 5.3) is a greedy heuristic approximation. Deriving a theoretically optimal policy — via Monte Carlo Tree Search, point-based value iteration, or Bayesian optimization over the belief state — would replace this approximation with a formally grounded solution and enable rigorous analysis of the exploration–exploitation tradeoff in autonomous security testing contexts.

7.4.2 Deterministic–AI Hybrid Architecture

The most impactful near-term research direction is a hybrid system combining deterministic CVE scanning (for high-confidence template matching) with PHANTOM’s semantic reasoning for novel vulnerability discovery. A confidence-gated handoff proto-

col — invoking LLM reasoning only after deterministic templates are exhausted on a given surface — would address Gaps G1 and G6 jointly while minimizing operational cost.

7.4.3 Multi-Model Routing

Different agent decision classes have fundamentally different computational requirements: payload synthesis requires breadth; strategic hypothesizing requires deep contextual reasoning. Routing these classes to specialized models — compact fast models for synthesis; frontier reasoning models for strategy — aligns with the mixture-of-experts paradigm and offers substantial improvements in both cost efficiency and decision quality within the same framework.

7.4.4 Cross-Scan Persistent Learning

The Correlation Engine currently initializes fresh Beta priors for each scan session. Implementing cross-scan persistence — accumulating learned priors from historical scans indexed by target technology stack and application type — would endow the system with institutional memory, enabling progressive improvement of chain-opportunity recognition as operational experience accumulates.

7.4.5 Formal Safety Verification

The seven-layer security model is validated through adversarial testing. Applying model-checking techniques to the scope guard, tool firewall, and RBAC modules would produce machine-verified proofs of the critical safety properties — no unauthorized network egress, no unauthorized tool invocation, no budget overrun — required for responsible deployment beyond controlled research environments.

7.4.6 Extension to Network and Infrastructure Targets

The current framework targets web application penetration testing. Extending the Hypothesis Ledger schema, Correlation Engine chain patterns, and Attack Graph node taxonomy to network infrastructure, Active Directory environments, and cloud-native attack surfaces would generalize the conceptual contribution to the broader autonomous offensive security domain.

7.5 Concluding Remarks

Penetration testing has historically required expert human judgment at each stage of the adversarial reasoning process: forming hypotheses about vulnerable surfaces, selecting and constructing payloads, interpreting ambiguous responses, and chaining individual findings into high-impact multi-step exploits. The prevailing assumption has been that this judgment is irreducibly human — that automation can perform reconnaissance and signature matching, but not reason.

This thesis challenges that assumption at the architectural level. By providing a language model with the structured scaffolding it systematically lacks — external belief-state management, evidence-weighted hypothesis scheduling, Bayesian chain-opportunity recognition, and a formally bounded execution environment — it becomes possible to instantiate a reasoning process that is functionally analogous to expert hypothesis-driven investigation.

The framework established here — gaps formally characterized, solutions architecturally grounded, contributions individually specified — constitutes a foundation upon which future autonomous security research can be built and measured. The six future directions outlined above are concrete, formally motivated, and tractable extensions of the theoretical framework established in Chapter 4. That transition, from open-ended aspiration to a well-posed and formally grounded research program, is itself a contribution of this work.

References

Bibliography

- [1] Alchourrón, C. E., Gärdenfors, P., and Makinson, D. (1985). On the logic of theory change: Partial meet contraction and revision functions. *Journal of Symbolic Logic*, 50(2):510–530.
- [2] Bishop, M. (2003). *Computer Security: Art and Science*. Addison-Wesley Professional, Boston, MA.
- [3] Deng, G., Liu, Y., Mayón, L., Liu, Y., Zhang, T., Liu, Y., Wang, H., Rao, Z., and Liu, Y. (2024). Pentestgpt: An llm-empowered automatic penetration testing tool. *arXiv preprint arXiv:2308.06782*.
- [4] Díaz-Verdejo, J., Málaga, M., Pereñíguez, F., and García-Teodoro, P. (2022). On the evaluation of dynamic analysis tools for security assessment of web applications. *IEEE Access*, 10:64874–64887.
- [5] Hagberg, A. A., Schult, D. A., and Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy2008)*, pages 11–15.
- [6] Happe, A. and Cito, J. (2023). Getting pwned by AI: Penetration testing with large language models. In *Proceedings of the 31st ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2023)*, pages 2082–2086. ACM.
- [7] ISC2 (2023). *Cybersecurity Workforce Study 2023*. ISC2 Research Report.
- [8] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., and Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38.
- [9] Kaelbling, L. P., Littman, M. L., and Cassandra, A. R. (1998). Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1-2):99–134.
- [10] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*.

- [11] Liu, R., Yang, Z., Li, C., and Yang, H. (2024). Evaluating LLMs for privilege-escalation scenarios. In *Proceedings of the 33rd USENIX Security Symposium*.
- [12] Nygard, M. T. (2007). *Release It! Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, Raleigh, NC.
- [13] OWASP Foundation (2021). *OWASP Top 10 — 2021*. Open Web Application Security Project. Available at: <https://owasp.org/Top10/>
- [14] Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., and Gonzalez, J. E. (2023). MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*.
- [15] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*.
- [16] Weidman, G. (2014). *Penetration Testing: A Hands-On Introduction to Hacking*. No Starch Press, San Francisco, CA.
- [17] Wooldridge, M. and Jennings, N. R. (1995). Intelligent agents: Theory and practice. *The Knowledge Engineering Review*, 10(2):115–152.
- [18] Xu, X., Feng, Z., Ye, J., Shen, Z., Cheng, Y., Zhang, H., Li, J., Lou, J., Lin, Q., Qiu, J., and Liu, Y. (2024). AutoAttacker: A large language model guided system to implement automatic cyber-attacks. *arXiv preprint arXiv:2403.01038*.
- [19] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2022). ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.
- [20] Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, K., Cheng, X., Bisk, Y., Fried, D., Alon, U., and Neubig, G. (2024). WebArena: A realistic web environment for building autonomous agents. In *Proceedings of ICLR 2024*.